

P.R.A.V.O.: Retrieval-Augmented Review of Russian Employment Contracts Against Labor Law

Course Project Report

Abstract

This report describes P.R.A.V.O. (Platform for Review and Analysis of labor contracts for Violations of Obligations), a project for finding legal matches and potential contradictions between clauses of Russian employment contracts and provisions of the Labor Code of the Russian Federation. The system combines document parsing, retrieval-augmented legal matching, natural language inference (NLI), definition lookup, rule-based document-level checks, and an interactive Streamlit interface. The implemented pipeline converts the Labor Code from DOCX into structured legal chunks, builds FAISS indexes with multilingual E5 embeddings, extracts contract clauses from PDF files with page coordinates, retrieves and reranks relevant legal norms, and classifies clause–norm pairs as entailment, neutral, or contradiction with a Russian NLI model. In addition to clause-level matching, the system checks mandatory or conditional contract requirements and supports manual annotation of results. The project demonstrates a practical hybrid architecture for legal NLP in a low-resource, domain-specific setting where exact legal links, interpretability, and user correction are more important than fully automated decision-making.

1 Introduction

Employment contracts are highly structured legal documents, but they are still written in natural language and often contain incomplete fields, template remnants, or clauses that worsen the employee’s position compared with statutory labor guarantees. Manual review is time-consuming because a reviewer must connect each contract clause to relevant provisions of the Labor Code and then decide whether the clause is compliant, merely related, or contradictory. This project addresses that workflow for Russian employment contracts.

P.R.A.V.O. is designed as an assistive legal NLP system rather than an autonomous legal authority. Given a PDF employment contract, it extracts numbered clauses, searches for matching Labor Code provisions, estimates the semantic relation between a clause and a provision, and displays the evidence in an interactive interface. The system follows a retrieval-augmented NLP pattern: a compact model retrieves evidence first, and a later stage reasons over the retrieved text. The system also performs document-level checks for mandatory contract elements, such as parties, workplace, labor function, salary terms, start date, social insurance, signatures, and required copies.

The main contribution of the project is an end-to-end

retrieval-augmented pipeline tailored to Russian labor law. The implementation combines four ideas:

1. The Labor Code is represented as structured chunks preserving article, part, subpart, and hierarchy metadata.
2. Contract clauses are parsed from PDF while retaining page bounding boxes, enabling visual highlighting in the original document.
3. Retrieval is separated from NLI classification: FAISS and a cross-encoder identify candidate legal norms, while the NLI model predicts whether the pair is entailment, neutral, or contradiction.
4. Definitions and mandatory-field checks are handled explicitly, because legal review often depends on terminology and formal completeness in addition to semantic similarity.

2 Task Definition

For each clause in an employment contract, the system seeks a small ranked set of relevant Labor Code chunks. Each candidate pair consists of a contract clause c and a legal norm n . The retrieval task is to rank norms by legal relevance:

$$\text{rank}(n \mid c).$$

The NLI task assigns probabilities to three legal relation labels: entailment, neutral, and contradiction. The most important risk signal is the contradiction probability:

$$s_{\text{contr}}(c, n).$$

The output is not only a label. It includes the matched norm text, article metadata, hierarchy path, retrieval similarity, optional reranker score, three NLI probabilities, document coordinates, and a user-facing automatic label.

At the document level, the task is different: the system checks whether the contract contains required information. Each requirement is represented by a set of regular expressions and, for conditional requirements, condition patterns. A requirement can be `present`, `missing`, or `not_applicable`. This layer captures formal obligations that are difficult to formulate as pairwise contradiction detection.

3 Data and Resources

3.1 Labor Code Corpus

The legal corpus is the DOCX version of the Labor Code of the Russian Federation. The preprocessing pipeline parses outline-level headings into a legal hierarchy: code part, section, chapter, and article. Article contents are then split into chunks at the level of article parts and subparts. The current repository contains 2,407 chunks in `artifacts/tk_chunks_new.jsonl`. Each chunk stores:

- a stable chunk identifier, such as `art:57:part:2:sub:1`;
- normalized and original norm text;
- article, part, and subpart numbers;
- the hierarchy path used for display and contextualization.

The choice of chunk granularity is important. Whole articles are often too long and cover multiple legal issues, while sentence-level splitting can lose legal context. Part/subpart chunks are short enough for vector search and NLI, but still contain complete legal obligations.

3.2 Definitions

Legal definitions are indexed separately from ordinary norms. The repository contains 80 definition entries in `data/definitions.json`. During analysis, the

Component	Implementation
PDF parsing	pdftext, PyMuPDF
Embedding model	multilingual E5 small
Vector store	FAISS via LangChain
Reranker	BGE reranker v2 m3
NLI model	RuBERT NLI three-way
UI	Streamlit

Table 1: Main implementation components.

system first searches definitions using literal alias matching and semantic FAISS retrieval. Definition chunk identifiers are excluded from ordinary norm retrieval so that definitional text does not dominate searches for enforceable rules. This separation improves interpretability: definitions are shown as terminology support, while norms remain the main evidence for compliance or contradiction.

3.3 Validation Data

For NLI evaluation, the project also uses a separate golden dataset, `data/golden_nli_dataset.csv`, with 114 labeled pairs. This golden dataset was partially generated with an LLM and then manually extended and corrected. It is balanced enough for course-project evaluation: 37 entailment, 36 neutral, and 41 contradiction examples.

4 System Architecture

The system follows a modular pipeline:

1. parse the Labor Code and build indexes;
2. parse the PDF contract into clauses;
3. run document-level requirement checks;
4. retrieve definitions and legal norms for each clause;
5. optionally rerank retrieved norms;
6. score contradictions with NLI;
7. display and annotate results in the UI.

The repository structure mirrors this pipeline. The `src/core/pdf` package handles PDF extraction and clause segmentation. The `src/core/rag` package builds legal chunks and indexes. The `src/core/retrieve` package performs FAISS search, definition retrieval, and reranking. The `src/core/classification` package contains the NLI scorer. The `src/core/document_checks` package implements requirement checks. The `src/ui` package pro-

vides the Streamlit application and service layer.

5 Document Processing

5.1 PDF Clause Extraction

Contract analysis begins with a PDF parser that extracts text lines with page coordinates. Lines are grouped into paragraphs and sections, and sections are split into numbered points using patterns such as 1., 1.1., and 2.3.. Nested subpoints are flattened: a child point receives the context of its parent introduction, which helps retrieval because isolated subpoints often omit the legal subject.

Each resulting `PdfChunk` contains a section heading, point number, original text, normalized text, and `page_to_bbox`. The bounding boxes make it possible to highlight contract clauses in the PDF viewer. The cleanup stage removes the preamble and service sections such as addresses, requisites, and signatures, because those sections tend to be poor inputs for clause-level legal matching. For final provisions, the parser also removes the signature tail when it begins to contain signature-related boilerplate.

5.2 Normalization

Text normalization is intentionally conservative. It lowercases text, removes unsupported punctuation, normalizes underscores and whitespace, and preserves common separators such as periods and commas. For legal retrieval, preserving some punctuation is useful because numbers, abbreviations, and clause boundaries can affect meaning. For the E5 model, the embedding wrapper adds the required `passage:` prefix to legal chunks and the `query:` prefix to contract clauses.

6 Retrieval and Reranking

The first-stage retriever uses `intfloat / multilingual-e5-small` embeddings, following the E5 query/passage convention, and a FAISS vector index. The FAISS distance is converted to an approximate cosine-like similarity by

$$\text{sim} = 1 - \frac{d}{2},$$

with clipping to the interval $[-1, 1]$. This representation is easier to display to users than raw L2 distance.

Because dense retrieval can return semantically similar but legally weak matches, the system optionally applies a cross-encoder reranker. The configured reranker is `BAAI/bge-reranker-v2-m3`. Instead of retrieving exactly k candidates, the service retrieves a larger candidate pool controlled by a multiplier, then reranks query–norm pairs and keeps the top k . The final result preserves both the original retrieval similarity and the reranker score, which is useful for debugging and for future calibration.

The retrieval layer also excludes definition chunks from ordinary legal norm matching. Definitions are useful evidence, but they should not replace article parts that impose rights, obligations, prohibitions, or guarantees. This design decision reflects a legal-domain distinction that a generic vector search pipeline would likely miss.

7 NLI Classification

The NLI layer is implemented with `cointegrated / rubert-base-cased-nli-threeway`, a Russian transformer model with three output labels: entailment, neutral, and contradiction. For every retrieved norm with available text, the system forms a pair consisting of the contract clause and the legal norm. The classifier returns a full probability distribution, not only a contradiction score.

The scorer is bidirectional by default. It computes all three label probabilities for (c, n) and (n, c) , then averages each label independently:

$$p_y(c, n) = \frac{p_y(c, n) + p_y(n, c)}{2}, \quad y \in Y.$$

Here $Y = \{\text{entailment}, \text{neutral}, \text{contradiction}\}$. This is a pragmatic choice. NLI models are directional, but legal comparison between a contract clause and a statutory norm is often closer to a symmetric relation from the user’s perspective. Bidirectional averaging makes the scores less sensitive to which text is treated as premise or hypothesis.

The automatic label is selected by `argmax` over the averaged probabilities. The UI stores and displays `entailment_score`, `neutral_score`, and `contradiction_score`; this makes the output more informative than a binary threshold. For example, a clause can be marked as neutral even when its contradiction score is nonzero, which is useful for legal review

because many retrieved provisions are related but not directly entailed or violated.

8 Document-Level Checks

Some legal requirements are not naturally expressed as contradiction pairs. For example, Article 57 of the Labor Code requires that a contract include information about the workplace, labor function, start date, salary terms, working conditions, social insurance, and other elements. Absence of these fields is a document-level defect, not a contradiction with a specific clause.

P.R.A.V.O. therefore includes a rule-based requirement checker. The repository currently defines 18 requirements in `data/contract_requirements.json`. Each requirement has an identifier, title, description, source chunk links, requirement type, and regex patterns. Conditional requirements also include condition patterns. For example, a fixed-term contract requirement becomes applicable only when the full contract text contains indicators of a fixed-term agreement.

The checker returns matched clauses and detects placeholder-like text such as empty quotes, underscores, empty parentheses, or phrases equivalent to “to be specified”. This is useful for template contracts: a clause may be present syntactically but still incomplete.

9 User Interface

The Streamlit UI connects the pipeline to an interactive review workflow. A user uploads a PDF, starts analysis, and receives:

- a PDF page viewer with highlighted clauses;
- cards with retrieved legal norms and article numbers;
- synchronized focus between the document and result cards;
- a checklist of document-level requirements;
- definition hints for legal terms;
- manual labels for contradiction review.

The UI caches expensive resources such as FAISS indexes, the reranker, and the NLI pipeline. This is necessary because transformer models and vector stores are too heavy to reload for every interaction. Manual annotations are saved as JSON with the analysis payload and user labels, making the interface both a review tool and a data collection tool.

Class / aggregate	Support	F1	ROC-AUC
Entailment	37	0.603	0.885
Neutral	36	0.681	0.842
Contradiction	41	0.622	0.864
Macro	114	0.635	0.863

Table 2: NLI metrics on the golden dataset.

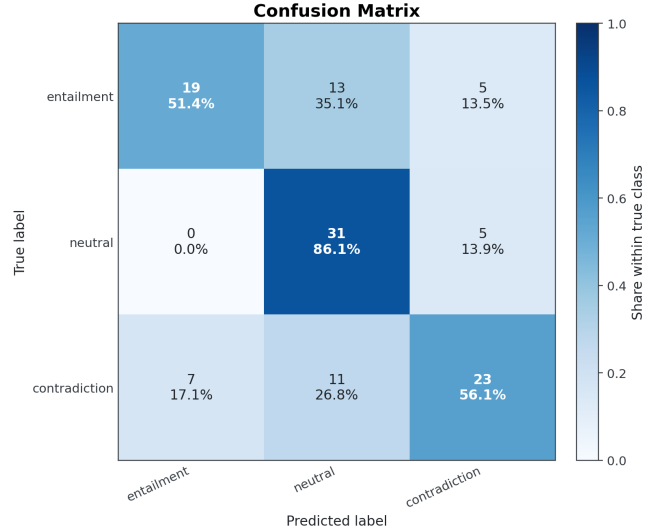


Figure 1: NLI confusion matrix on the golden dataset.

10 Evaluation

The current project is best evaluated at three levels. First, PDF parsing can be checked by inspecting whether numbered clauses and bounding boxes align with the original document. Second, retrieval can be evaluated with recall@k or mean reciprocal rank over annotated clause-norm pairs. Third, contradiction scoring can be evaluated with precision, recall, and F1 over labeled NLI pairs.

The NLI evaluation reported here uses `data/golden_nli_dataset.csv`. The evaluation mode is `bidirectional_average_argmax`: the model predicts both text orders, averages class probabilities, and selects the highest-probability label. The resulting accuracy is 0.640. Macro F1 is 0.635 and macro one-vs-rest ROC-AUC is 0.863.

The confusion matrix shows that the model is strongest on neutral examples: 31 of 36 neutral cases are classified correctly. Entailment examples are often confused with neutral cases, and contradiction examples are split between correct contradiction predictions and neutral predictions. This pattern is acceptable for an assistive

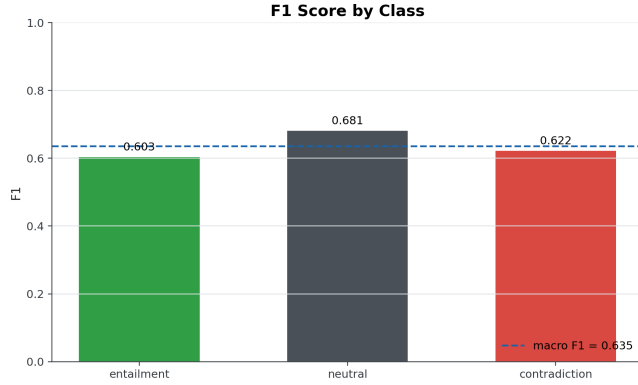


Figure 2: F1 score by NLI class.

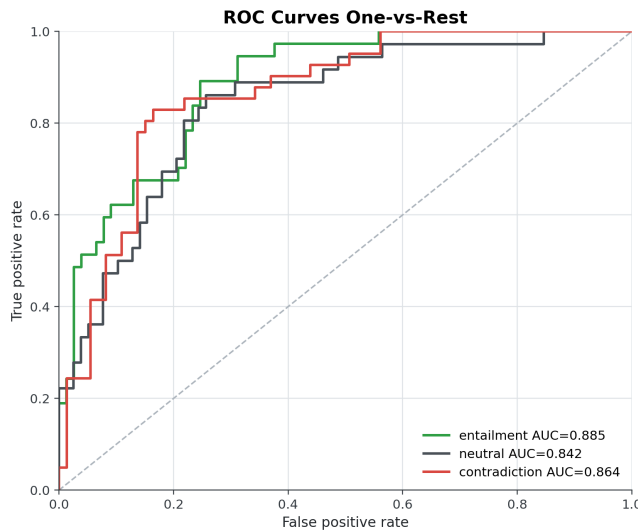


Figure 3: One-vs-rest ROC curves for NLI labels.

review tool but indicates that the model should not be treated as a final legal decision maker.

Qualitative inspection shows several expected strengths. Near-duplicate contract clauses that restate the Labor Code are matched accurately because the E5 retriever captures semantic paraphrase and the reranker can compare the full pair. The system is also effective for common employment-law violations that have clear numeric or deontic signals: vacation below 28 days, working time above 40 hours per week, salary paid once per month, or the absence of written consent where the law requires it.

The main failure modes are also predictable. Broad contract clauses can retrieve general legal definitions rather than enforceable norms, which motivated the separate definition layer. Some legal contradictions depend

on external facts, such as worker category, workplace conditions, or employer type. Other contradictions require arithmetic or temporal reasoning that generic NLI models may not handle reliably. For example, comparing salary to the current minimum wage requires up-to-date external data, not only text similarity.

11 Limitations

The system has several limitations. First, it depends on the quality of PDF text extraction. Scanned documents, unusual layouts, or broken numbering can reduce clause segmentation quality. Second, retrieval is only as good as the chunking strategy and embedding model. A relevant legal rule may be missed if the query is vague or if the norm is split across multiple chunks. Third, the NLI model was not trained specifically for Russian labor-law contradiction detection. Its scores should therefore be interpreted as heuristic risk indicators, not legal conclusions.

Fourth, the requirement checker is rule-based. This makes it transparent and easy to edit, but it can miss paraphrases or produce false positives when patterns match boilerplate. Fifth, the current evaluation data is small and partly synthetic. A stronger evaluation would require a larger set of real contracts annotated by legal experts, with separate labels for relevance, contradiction, incompleteness, and uncertainty.

12 Future Work

Several improvements would make the project stronger. A first step is a reproducible evaluation script that reports recall@k for retrieval, reranker ablations, and NLI precision/recall/F1. A second step is legal-context augmentation: the NLI model could receive article titles or chapter titles when they clarify the rule. A third step is specialized checks for numeric and temporal obligations, such as salary frequency, vacation length, probation duration, notice periods, and overtime limits. These checks could combine regex extraction with deterministic comparison rather than relying only on NLI.

Another direction is active learning. Since the UI already supports manual labeling, user corrections can be converted into new training or validation examples. Over time, this would produce a domain-specific dataset of contract clauses, matched Labor Code provisions, and ex-

pert labels. Finally, deployment should include stronger safeguards: versioning of the Labor Code corpus, explicit model versions, confidence calibration, and clear disclaimers that the tool provides decision support rather than legal advice.

13 Conclusion

P.R.A.V.O. demonstrates a complete retrieval-augmented workflow for reviewing Russian employment contracts against the Labor Code. The project integrates legal-document parsing, structured statutory chunking, semantic retrieval, reranking, NLI contradiction scoring, definition lookup, rule-based document checks, and an interactive annotation interface. Its design reflects the practical constraints of legal NLP: evidence must be traceable, results must remain inspectable, and automation should support rather than replace human review.

Although the current evaluation resources are modest, the architecture is extensible. The system already contains the data structures needed for quantitative evaluation and iterative improvement, and the golden NLI evaluation provides a concrete baseline for future model changes. As a course project, it provides a concrete implementation of applied NLP for legal compliance analysis and a foundation for future work on more robust Russian legal reasoning.